COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE031	**SECTION:**	CPE21S1
DATE PERFORMED:	JULY 28, 2026	**DATE SUBMITTED:**	AUGUST 1, 2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Quejado

ACTIVITY NO. 4 **Stacks (PROGRESS REPORT 2)**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create a stack using the C++ STL

CODE

```cpp
#include <iostream>
#include <stack>
// commands (so far)
/*
    .empty
    .push
    .pop
    .swap
    .top
    .size
*/

int main(){

    std::stack<int> newStack;

    std::cout << "Stack Empty? " << newStack.empty() <<"\n";

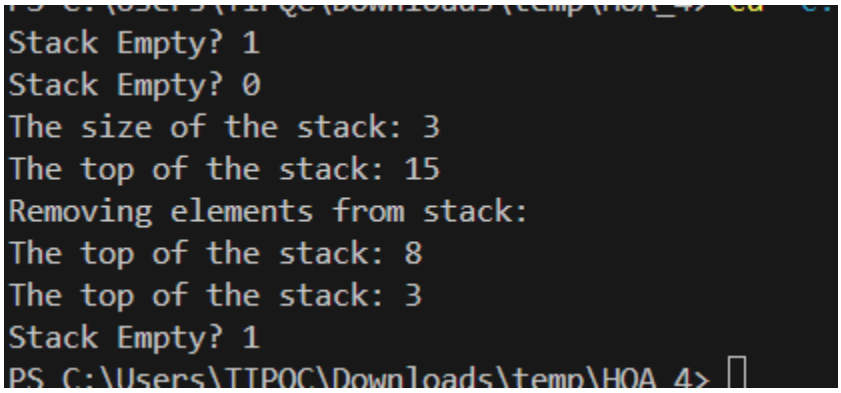
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout << "Stack Empty? " << newStack.empty() <<"\n";
    std::cout << "The size of the stack: " << newStack.size() <<"\n";
    std::cout << "The top of the stack: " << newStack.top() <<"\n";

    std::cout << "Removing elements from stack: \n";
    newStack.pop();
    std::cout << "The top of the stack: " << newStack.top() <<"\n";
    newStack.pop();
    std::cout << "The top of the stack: " << newStack.top() <<"\n";
    newStack.pop();
    std::cout << "Stack Empty? " << newStack.empty() <<"\n";


    return 0;
}
```

OUTPUT


Stack Empty? 1
Stack Empty? 0
The size of the stack: 3
The top of the stack: 15
Removing elements from stack:
The top of the stack: 8
The top of the stack: 3
Stack Empty? 1
PS C:\Users\TIPQC\Downloads\temp\HOA_4>

ILO B1: Stacks using array

CODE

```cpp
#include <iostream>

void push();
void pop();
void top();
bool empty();
bool full();

const size_t maxCap = 100;

int stack[maxCap];
int Top = -1;
int i, newData = 0;

int main(){

    int choice;

    std::cout << "Enter the number of max element for stack: ";
    std::cin >> i;

    while(true){
        std::cout << "Stack operations\n";
        std::cout << "1. PUSH\n2. POP\n3. TOP\n4. EMPTY\n5. END PROGRAM\n";
        std::cin >> choice;

        switch(choice){
            case 1:
            push();
            break;

            case 2:
            pop();
            break;

            case 3:
            top();
            break;

            case 4:
            empty();
            break;

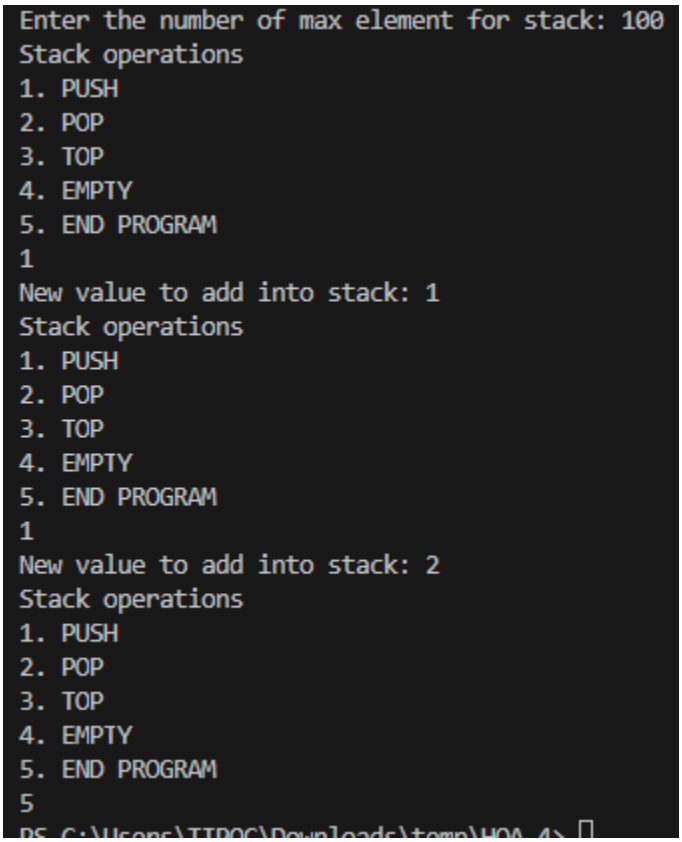
            case 5:
            return 0;

            default:
            std::cout << "Invalid Choice\n";
        }
    }

    return 0;
}
```

```
55  bool empty(){
56      if (Top == -1){
57          return true;
58      }
59      else {
60          return false;
61      }
62  }
63
64  bool full(){
65      if ( Top >= i-1){
66          return true;
67      }
68      else {
69          return false;
70      }
71  }
72
73  void push(){
74      if (full()) {
75          std::cout << "stack overflow";
76          return;
77      }
78      std::cout << "New value to add into stack: ";
79      std::cin >> newData;
80      stack[++Top] = newData;
81  }
82
83  void top(){
84      if(empty()){
85          std::cout << "Stack underflow\n";
86          return;
87      }
88
89      std::cout << "The element on the top of stack is: " <<stack[Top] << std::endl;
90
91  }
92
93  void pop(){
94      if(empty()){
95          std::cout << "Stack underflow\n";
96          return;
97      }
98
99      std::cout << "Popping: " << stack[Top] << std::endl;
100     Top--;
101 }
```

OUTPUT



ANALYSIS:

I'm familiar with tampering with arrays, I treat it mostly like data banks. Using arrays as stacks helps me visualize how it works and stores information. "Pushing" is putting elements on top of what's already been put, but isn't it better to call it "Putting"? Anyways, "Popping" is removing what's on top so the most recent element that has been put in the stack is the first one to go out, hence the acronym "LIFO". Lastly, the boolean values "empty" and "full" are for checking if the stack is gonna overflow or is currently empty.

CODE

New function for printing the stack from top to bottom

```
108  void print(){
109
110      if(empty()){
111          std::cout << "No stack to print\n";
112          return;
113      }
114
115      else{
116
117          for (int j = Top; j != -1; j--){
118          std::cout << stack[j] << ", ";
119      }
120      std::cout << std::endl;
121      }
122
123  }
```

OUTPUT

```
Stack operations
1. PUSH
2. POP
3. TOP
4. EMPTY
5. END PROGRAM
6. PRINT
6
1, 2, 3,
```

ANALYSIS:

When it comes to printing an array, my first initial thought would be using a for-loop. The difference here is that the array can be any size and still not be full or can be empty. This is where the “Top” variable comes into play, since it holds the amount of elements in the array in index value, we can use it as an anchor to count from the top to the bottom of the stack. Here in the output, I pushed the elements 3,2,1 in this order and it printed out 1,2,3. This is because I made the for-loop count down from the “Top” value until it reaches -1 which indicates that we’ve reached the end of the stack. It’s important that we start from the “Top” so that we don’t print garbage values as well.

ILO B2: Stack using Linked List

CODE

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H


template<typename T>
class Node{
    public:
    T data;
    Node* next;
};

template<typename T>
Node<T>* head = nullptr;


template<typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
void push (T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

template<typename T>
T pop (){
    if (head<T> == nullptr){
        std::cout << "stack undeflow\n";
        return T();
    }

    Node<T>* temp = head<T>;
    T temVAL = temp->data;

    head<T> = head<T>->next;
    delete temp;
    temp = nullptr;
    return temVAL;

};

template<typename T>
void Top(){
    if (head<T> == nullptr){
        std::cout << "stack underflow\n";
        return;
    }
    std::cout << "The top is: " << head<T>->data << "\n";
};
```

```cpp
template<typename T>
void print(){

    if (head<T> == nullptr){
        std::cout << "stack underflow\n";
        return;
    }

    Node<T>* temp = head<T>;
    while (temp != nullptr) {
        std::cout << temp->data << ", ";
        temp = temp->next;
    }
    std::cout << "\n";

}

#endif
```

Main driver function:

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout << "--- pushing 'c' 'p' 'e' onto stack --- \n";

    push('c');
    push('p');
    push ('e');

    Top<char>();


    std::cout << "\n --- printing all elements --- \n";
    print<char>();

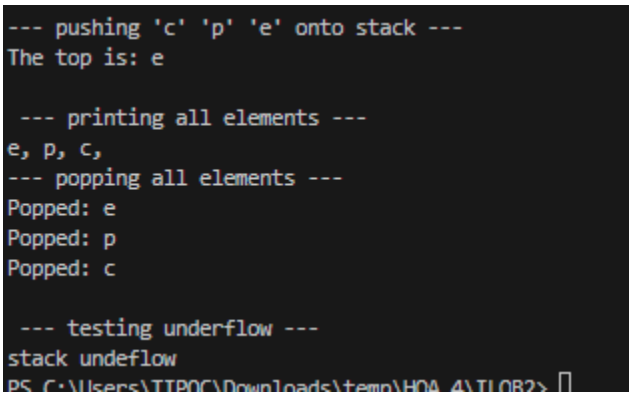


    std::cout << "--- popping all elements --- \n";
    while(!isEmpty<char>()){
        std::cout << "Popped: " << pop<char>() << std::endl;
    }


    std::cout << "\n --- testing underflow --- \n";
    pop<char>();


    return 0;
}
```

OUTPUT



ANALYSIS

Here is my analysis for each operation in the h. File…

The class **Node** lets us create our nodes with its data and the pointer variable for next.

Now we set the **head** as a global variable for the top of our stack.

The **isEmpty** function returns a boolean value whether the stack is empty or not.

The **push** function, every time we add a new node we reassign the head to the new node.

For this situation, the **pop** function returns the data that has been popped from the stack.

The **Top** function prints out what is currently at the top/head of the stack

The **print** function sets a temporary pointer variable to the head so that we can print everything without changing its orientation.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line

Steps:
1. Create a Stack.
2. While(end of input is not reached) {
    a) If the character read is not a symbol to be balanced, ignore it.
    b) If the character is an opening symbol, push it onto the stack.
    c) If it is a closing symbol:
        i) Report an error if the stack is empty.
        ii) Otherwise, pop the stack.
    d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

ILO C:Solve problems using implementation of stack

CODE

Iloc.h file code

```
#ifndef ILOC_H
#define ILOC_H

template<typename elmo>
class Node{
    public:
    elmo data;
    Node* next;
};

template<typename elmo>
Node<elmo>* head = nullptr;

template<typename elmo>
bool isEmpty(){
    return head<elmo> == nullptr;
}

template <typename elmo>
void push (elmo newData){
    if (newData != '(' && newData != '{' && newData != '['){
        return;
    }

    Node<elmo>* newNode = new Node<elmo>;
    newNode->data = newData;
    newNode->next = head<elmo>;
    head<elmo> = newNode;
}

template<typename elmo>
elmo pop(){
    if (isEmpty<elmo>()){
        std::cout << "stack undeflow\n";
        return elmo();
    }

    Node<elmo>* temp = head<elmo>;
    elmo temVAL = temp->data;

    head<elmo> = head<elmo>->next;
    delete temp;
    temp = nullptr;
    return temVAL;

}
```

```cpp
template<typename elmo>
void Top(){
    if (isEmpty<elmo>()){
        std::cout << "stack undeflow\n";
        return;
    }
    std::cout << head<elmo>->data;
}

template<typename elmo>
void print(){

    if (head<elmo> == nullptr){
        std::cout << "stack underflow\n";
        return;
    }

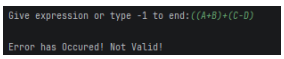
    Node<elmo>* temp = head<elmo>;
    while (temp != nullptr) {
        std::cout << temp->data << ", ";
        temp = temp->next;
    }
    std::cout << "\n";

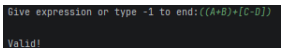
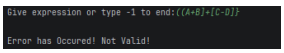
}



#endif #ifndef ILOC_H
```

SELF CHECKING:

OUTPUT			
Expression	**Valid? (Y/N)**	**Output (Console Screenshot)**	**Analysis**
(A+B)+(C-D)	Y		So ignoring all the non-balancing characters, it'll push the first "(" then when it comes to the ")", it'll pop the top which is the "(". Then, it'll see that it's the matching character for the closing delimiter and it will not return an error. This process gets repeated one last time since it's the same open and close delimiters. Therefore, a valid expression
((A+B)+(C-D)	N		Now, it pushes the first two "(" and when it comes to the first ")", it'll pop the top and not return an error. Then it'll push the third "(" and read the ")" after. It will pop the top and then return not an error, but the code checks if the stack is empty at the very end, since it isn't it'll return an error so it is an invalid expression.

((A+B)+[C-D])	Y		It'll push the first two "(" and then pop the top because of the ")" and not return the error. The program then pushes "[" to the stack and pops it again after reading the "]". Lastly, it'll detect the remaining ")" and pop the last remaining data in the stack which was the first "(". This then outputs that the expression is valid.
((A+B]+[C-D]}	N		In this last expression, it returned an error very quickly since after pushing the first two "(", it reads the "]". But when it popped the top of the stack which is "(", it doesn't match "]". So, with this mismatch, it returned as an invalid expression.

CONCLUSION/OBSERVATION

After completing all the activities, it has given me a bit of an information overload. For the procedures, I was doing fine in understanding and implementing it. When it came to the supplementary, I was taken aback. I did not have a single clue how to implement stacks using arrays AND linked lists in one program, it's hard for me to visualize even if I tried drawing it. So in conclusion, I just used an array to store my expression inputs as characters and push each character in a stack using a linked list. I got lost in the logic in my coding so I asked Gemini for assistance. All in all it was a very confusing but enjoyable experience

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Logic and Sequence of Code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explains the logic of the "if-else" statements in my code and the sequence**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.